大题一：CNN原理

考点1：卷积计算

input

a	b	c	d
e	f	g	h
i	j	k	l

kernal

w	x
y	z

→ output

aw+bx+ ey+fz	bw+cx+ fy+gz	cw+dx+ gy+hz
ew+fx+ iy+jz	fw+gx+ jy+kz	gw+hx+ ky+lz

考点2：给定卷积核层数要和输入图像层数保持一致

有n个卷积核，就输出n层，输出图像大小用公式计算。

$$output_size = \left\lfloor \frac{input_size + 2 \times padding - kernel_size}{stride} \right\rfloor + 1$$

$\lfloor \cdot \rfloor$：表示向下取整。

例1：有一个224×224的，卷积核3×3，步长stride为2，填充padding为1，则输出为 $\left\lfloor \frac{224+2\times1-3}{2} \right\rfloor + 1 = \left\lfloor \frac{223}{2} \right\rfloor + 1 = 111+1 = 112$

input：32×32×3，10个5×5的filters，stride=1，pad的多少

求output_size.

$$output_size = \frac{(32+2\times2-5)}{1}+1 = 32.$$

所以输出 32×32×10.

考点2：求参数大小.

按上一个例子：则参数大小 = 10 × (5×5×3 + 1) = 760

- 10：卷积核个数
- 5×5×3：卷积核大小
- 1：偏置参数

考点4：池化层解释.

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

最大池化，2×2 ⇒

6	8
3	4

平均池化，2×2 ⇒

↳ 结果是浮点数，不会四舍五入或向下取整为2维了.

大题型二、关联规则挖掘步骤

1、什么是支持度？什么是可信度？

支持度是超项集X和Y在数据库D中同时出现的概率。

$$sup = P(X \cup Y) = \frac{count(X \cup Y)}{Total\ Transactions}$$

可信度是超项集X出现的情况下，项集Y在数据库D中同时出现的条件概率

$$conf = P(Y|X) = \frac{P(X \cup Y)}{P(X)} = \frac{support(X \cup Y)}{support(X)}$$

总：支持度用于确定频繁项集，可信度(置信度)用于确定关联规则。

2、什么是频繁项集？

满足最小支持度阈值的所有项集称为频繁项集。

3、使用Apriori算法找频繁项集（见后面例题）

4、使用FP-growth算法找频繁项集，并构造FP树（见后面例题）

5、Apriori算法和FP-growth算法的异同。

Apriori算法：逐层生成候选集，多次扫描数据库，候选集多时效率低。

FP-Growth：构建频繁模式树，无需候选集，两次扫描，适合稠密数据集

6、根据找到的频繁项集，确定关联规则.

(1). 生成候选规则：对于每个频繁项集L，生成所有可能的非空子集A和B，使得 $A \cup B = L$ 且 $A \cap B = \emptyset$，即规则形式为 $A \Rightarrow B$.

频繁二项集 {A, B} 只能生成两条可能的关联规则：$A \Rightarrow B$；$B \Rightarrow A$.

频繁三项集 {A, B, C}，可能的候选规则包括：

$A \Rightarrow \{B, C\}$；$B \Rightarrow \{A, C\}$. $C \Rightarrow \{A, B\}$

$\{A, B\} \Rightarrow C$， $\{A, C\} \Rightarrow B$； $\{B, C\} \Rightarrow A$.

(2) 然后计算每个规则的支持度和置信度，筛选关联规则.

例题.

有如下交易数据库，最小支持度是60%，最小可信度是75%.

① 请用Apriori算法找出所有的频繁项目集

② 请找出所有的关联规则

③ 请用FP-Growth算法找出频繁项，并据此画出相应的FP-tree.

交易号	交易商品集合
1	{O, P, Q, R, S, T}
2	{O, P, R, S}
3	{O, P, S}
4	{O, P, Q, Q}
5	{O, P, P}

①

共有5条交易，最小支持度=60%，即最少出现三次. 5×60%=3.

1项集的支持度如下：O:5；P:4；Q:2；R:2；S:3；T:1.

因此频繁1项集为：{O}，{P}，{S}.

在频繁1项集的基础上找2项集和3项集.

2项集的支持度如下：{O,P}:4；{O,S}:3；{P,S}:3.

因此频繁2项集为：{O,P}，{O,S}，{P,S}.

3项集的支持度为：{O,P,S}:3.

因此，最终频繁项集为{O}，{P}，{S}，{O,S}，{P,S}，{O,P,S}.

---

②找关联规则：找到1项集不为空，1项以上构成关联规则.

根据频繁2项集和3项集生成候选规则.

O→P，O→S，P→S；P→O；S→O；S→P.

~~{O,S}→P~~ {O,P}→S，{O,S}→P；{P,S}→O；

S→{O,P}，P→{O,S}；O→{P,S}.

分别计算以上规则的支持度和置信度，可以生成关联规则.

例如：O,P→S；支持度3；置信度=3/5=60%（不满足）.

O,S→P，支持度3；置信度=3/3=100%（满足）.

③ FP-Growth 算法与 FP-tree.

步骤：①. 计算软件1次集，不用计算软件2次和3次.

{O:5}；{P:5}；{S:3}.

②. 按支持度降序排序项：P, O, S.

③. 按此顺序重排每条交易：

交易	重排后
1	P, O, S
2	P, O, S
3	P, O, S
4	P, O
5	P, O

④. 构建FP-tree.

{}

↓

P:5

↓

O:5

↓

S:3.

FP-tree有一条链，因为所有交易都包含P和O，且S出现在前三条交易中.

贝叶斯定理：$P(AB)=P(A|B)*P(B)=P(B|A)*P(A)$.

$$P(h|D)=\frac{P(D|h)P(h)}{P(D)}.$$

的来自训练集.

题型三：分类算法：

例题：贝叶斯相关的分类算法。

算法1：使用朴素贝叶斯算法进行购买电脑的分类。

下表中是购买电脑的情况统计，训练集的特征包括年龄，收入，学生，信用，以及购买情况。

测试案例：一个收入中等、信用度良好的青年、学生潜在顾客，是否会购买电脑呢？

首先计算测试集中购买电脑的客户中不同属性的概率。

P(青年|购买) = 2/9 = 0.222；P(收入中等|购买) = 4/9 = 0.444

P(学生|购买) = 6/9 = 0.66；P(信用中等|购买) = 6/9 = 0.66

购买电脑的似然概率 = 0.222 × 0.444 × 0.66 × 0.66 = 0.044。

然后计算测试集中不同属性下不购买电脑的概率：

P(青年|不买) = 3/5 = 0.6；P(收入中等|不买) = 2/5 = 0.4

P(学生|不买) = 1/5 = 0.2；P(信用中|不买) = 2/5 = 0.4

不购买电脑的似然概率 = 0.6 × 0.4 × 0.2 × 0.4 = 0.019。

P(购买) = 9/14 = 0.643；P(不买) = 5/14 = 0.35。

因为 P(购买|X) = 0.044 × 0.643 = 0.028

P(不买|X) = 0.019 × 0.35 = 0.00

$\log_2 X$ 表示 2 的 n 次方等于 X，计算方法主要有以下两种

1. 换底公式：$\log_2 X = \frac{\ln X}{\ln 2}$ 或 $\frac{\lg X}{\lg 2}$。

2. 在科学计算器上，一般直接找 "$\log_2$" 键，如果没有，则用换底公式，先算自然对数再除以 $\ln 2$；

$$I(n_1, n_2) = -\frac{n_1}{n_1+n_2}\log_2\left(\frac{n_1}{n_1+n_2}\right) - \frac{n_2}{n_1+n_2}\log_2\left(\frac{n_2}{n_1+n_2}\right)$$

在 ID3 算法的每次迭代中，划分属性都需要重新计算信息熵和条件熵。因为下面三点：

① 数据集不同了，② 非根节点的计算特征

③ 不复用全局值。

算法2：使用2)、3)算法进行购买电脑决策。

在是否购买电脑这个决策中，9个买电脑，5个不买电脑，因此，D的熵为 $Info(D)=I(9,5)=-\frac{9}{14}\log_2\left(\frac{9}{14}\right)-\frac{5}{14}\log_2\left(\frac{5}{14}\right)=0.940$ bit.

对于年龄这个属性，青年有5人（2人买，3人不买）.

中年有4人（4人买，0人不买）

老人有5人（3人买，2人不买）.

计算年龄属性的条件熵.

$$Info_{age}(D)=\frac{5}{14}I(2,3)+\frac{4}{14}I(4,0)+\frac{5}{14}I(3,2)$$

$$=0.694 \text{ bit}$$

所以年龄这个属性的信息增益为：$Info(D)-Info_{age}(D)$

$=0.940-0.694=0.246$ bit.

分别计算其它属性的信息增益.

$Gain(income)=0.029$ bit

$Gain(fancy)=0.151$ bit

$Gain(credit_rating)=0.048$ bit.

选择年龄作为第一个根节点进行分类，然后在每个子树中，根据其

特征的信息增益量进行每个划分，递归地形成每个划分上的

样本判定树.

习题 5.5

使用[redacted]算法判断[redacted]对是否 Tensure 判断了

对 ID3 决策树算法的建树过程和决策过程进行描述.

建树过程：① 从训练数据中选择一个具有最好划分能力的特征作为根节点，ID3 算法通过计算特征的信息增益来选择

② 根据选定的特征划分数据集

③ 对每个子数据集递归建立子树，对每个子数据集，重复步骤①和步骤②，直到满足停止条件

④ 构建出完整的决策树

决策过程：① 从根节点开始，根据节点的特征判断新样本应该向左子树还是右子树移动

② 沿着选择的路径继续移动，直到达到叶节点

③ 根据叶节点的类别输出预测结果

使用朴素贝叶斯算法判断垃圾邮件.

已知训练集中垃圾邮件出现的概率为 $P(h+)=0.2$；训练集中正常邮件出现的概率为 $P(h-)=0.8$；

单词出现频率表如下表所示：

分词	垃圾邮件中出现的概率	正常邮件中出现的概率
免费	0.3	0.01
奖励	0.2	0.01
网站	0.2	0.2

一封邮件[redacted]中包含〈免费，奖励，网站〉是为垃圾邮件.

$$P(h+|D)=P(h+)*(P(D|h+))/P(D)$$
$$=0.2*(0.3*0.2*0.2)/P(D)=0.0096/P(D)$$

$$P(h-|D)=P(h-)*(P(D|h-))/P(D)$$
$$=0.8*(0.01*0.01*0.2)/P(D)=0.000016/P(D)$$

由贝叶斯后验概率，可得 $P(h+|D)>P(h-|D)$

则可以判断这封邮件是垃圾邮件

题型四：聚类算法

① 曼哈顿距离：$d(i,j)=|x_{i1}-x_{j1}|+|x_{i2}-x_{j2}|+\cdots+|x_{ip}-x_{jp}|$.

欧几里德距离：$d(i,j)=\sqrt{|x_{i1}-x_{j1}|^2+|x_{i2}-x_{j2}|^2+\cdots+|x_{ip}-x_{jp}|^2}$.

② 聚类方法的种类：(1) 划分方法：K-means 算法，
K-means 改进算法.
(K-mode 算法，用众数代替平均数；
K-means++ 算法，选离已有点为前提点；
K 中心点法.

(2) 层次方法：自底向上：AGNES 算法
自顶向下：DIANA 算法.

(3) 密度方法：DBSCAN，DENCLUE.
OPTICS.

③ 常用算法的优缺点：

K-means 算法步骤：(1). 从数据集中随机取 K 个样本作为初始聚类中心.
(2). 针对数据集中的每个样本 x_i，计算它到 K 个聚类中心的距离，并将其分到距离最小的聚类中心对应的类中.
(3) 针对每个类别 C_i，重新计算其聚类中心.
(4). 重复第 2 步和第 3 步，直到 $\sum_{i=1}^{k}\min_{x\in C_i}(\|x_j-c_i\|^2)$ 小于算法的阈值.

K-means的优点：①聚类时间短. ②当结果簇是密集时，簇间区别明显，效果较好. ③相对可扩展和有效，能对大数据集进行高效划分

缺点：①用户必须指定簇的个数
②常常终止于局部最优.
③只适用于数值属性数据（计算均值有意义）
④对噪声和异常数据比较敏感
⑤对于不同的初始值，结果可能不同
⑥不适合发现非凸面形状的簇.

K-means：参数个数就1个，为K，一般只能找出凸的形状.
对噪声敏感，不鲁棒.

DBSCAN：有ε，MinPts两个参数，可以找出任意形状的形状.
鲁棒性较高.

OPTICS：有ε，MinPts两个参数，可以找出任意形状.

OO DBSCAN更鲁棒.

例题一：

设某聚类算法的聚类结果如下图所示，第一个簇的5个对象分别为(3,4)，(4,5)，(2,6)，(3,8)，(4,7)。第二个簇的5个对象分别为(6,2)，(7,2)，(7,4)，(8,4)，(8,5)。请计算C1和C2之间的中心点距离，用欧几里得距离函数。

C1的中心点为 $\left\lfloor \frac{3+4+2+3+4}{5} \right\rfloor = 3$，

$\left\lfloor \frac{4+5+6+8+7}{5} \right\rfloor = 6$。 即(3,6)

C2的中心点为 $\left\lfloor \frac{6+7+7+8+8}{5} \right\rfloor = 7$，

$\left\lfloor \frac{2+2+4+4+5}{5} \right\rfloor = 3$。 即(7,3)

欧几里德距离为 $\sqrt[2]{(3-7)^2+(6-3)^2} = \sqrt[2]{25} = 5$。

例题二：

假设数据挖掘的任务是将如下的6个点(二维)聚类为三个类。

$A_1(3,9)$, $A_2(2,7)$, $A_3(6,9)$, $A_4(1,3)$, $A_5(4,9)$, $A_6(2,7)$

距离函数是曼哈顿函数。

(a). 假设初始选择 A_1, A_3 和 A_6 为每个聚类的中心，请用 k-means 算法给出在第一次循环执行后的聚类结果及中心。

(b). 分析并举例说明 k-means 算法的缺点，针对这些缺点，请设计一种改进的聚类算法，给出算法思路的描述。

(a). 计算采用如下步骤：

第一次迭代：

中心点	$A_1(3,9)$	$A_2(2,7)$	$A_3(6,9)$	$A_4(1,3)$	$A_5(4,9)$	$A_6(2,7)$
$A_1(3,9)$	(0)	(3)	3	(8)	1	(3)
$A_3(6,9)$	3	6	(0)	11	2	6
$A_5(4,9)$	1	4	2	9	(0)	4

例如：→ 计算曼哈顿距离 $= |3-2|+|9-7| = 1+2 = 3$。

聚类结果，每列找最小的数，它所在行就归为这个簇。

括号里面的值为最小值，

簇1为：$A_1(3,9)$，$A_2(2,7)$，$A_4(1,3)$，$A_6(2,7)$.

更新中心点均值：$\bar{x}=\frac{(3+2+1+2)}{4}=2$

$\bar{y}=\frac{(9+7+3+7)}{4}=6.5$.

向下取整为$(2,6)$.

（注意，曼哈顿距离的最优中心是中位数，部分简单题会用均值，为了准确，应当用中位数，但本题的较简单，用均值也可）

簇2为：$A_3(6,9)$，中心为$(6,9)$.

簇3为：$A_5(4,9)$，中心为$(4,9)$.

第2次迭代：

中心点	$A_1(3,9)$	$A_2(2,7)$	$A_3(6,9)$	$A_4(1,3)$	$A_5(4,9)$	$A_6(2,7)$
$(2,6)$	4	(1)	7	(4)	5	(1)
$A_3(6,9)$	3	6	(0)	11	2	6
$A_5(4,9)$	(1)	4	2	9	(0)	4

簇1为：$A_2(2,7)$，$A_4(1,3)$，$A_6(2,7)$，更新中心为：$(1,5)$.

簇2为：$A_3(6,9)$，更新中心为：$(6,9)$.

簇3为：$A_1(3,9)$，$A_5(4,9)$. 更新中心为：$(3,9)$.

(6). K-means 缺点及改进.

① 在更新中心时，采用算法平均，所以对噪声敏感，如1，2，100，平均为34，此时也可以改为众数 median.

② 需要预先输入K值，对于有些数据来说，我难预先知道K值，此时可以采用密度估计方法.